Algoritmos e Programação: Orientação a Objetos

Introdução a Orientação a Objetos

Profa. Andriele Busatto do Carmo

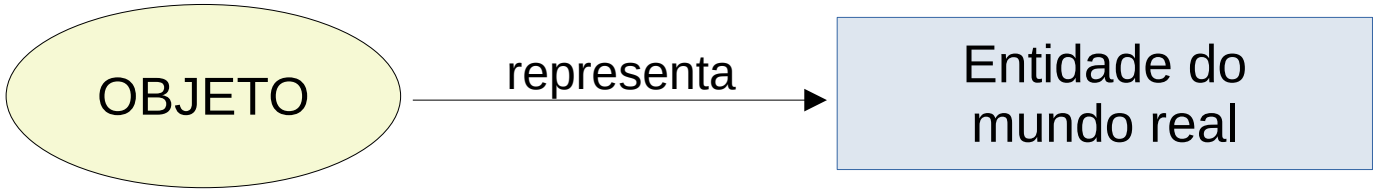
acarmo@unisinos.br

UNISINOS

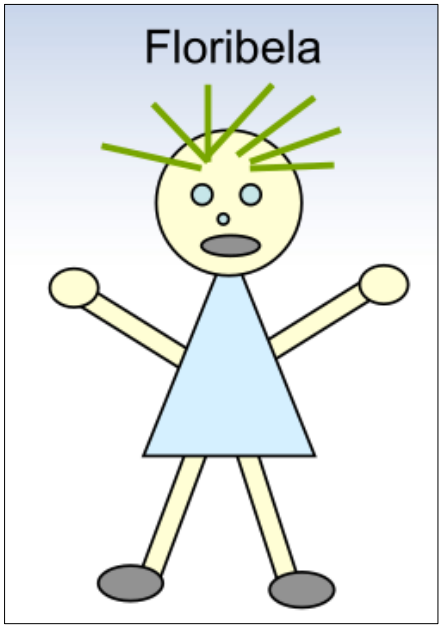
Orientação a objetos

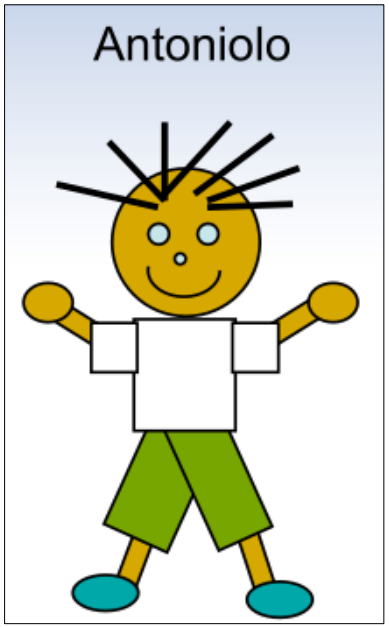
- Programação orientada a objetos:
  - Os problemas são pensados em termos de OBJETOS, suas propriedades e métodos.

- Para um sistema de uma biblioteca, por exemplo, teremos a divisão de objetos, tais como Livro, Andar, Matéria, Atendente, Usuário, etc.



Orientação a objetos





- Nome: Floribela
- Cor do cabelo: verde
- Cor da roupa: azul
- Cor do sapato: cinza
- Cor da pele: amarela
- Altura: 6cm
- Humor: assustada

- Nome: Antoniolo
- Cor do cabelo: preto
- Cor da roupa: verde e branca
- Cor do sapato: azul
- Cor da pele: marrom
- Altura: 5,5cm
- Humor: feliz

Orientação a objetos

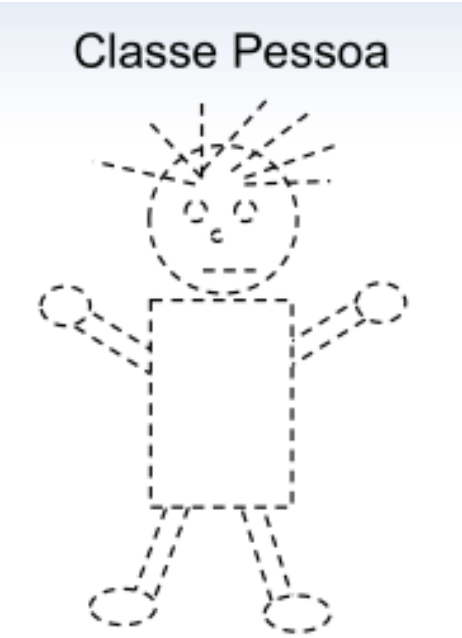
- Uma classe representa um conjunto de objetos que possuem determinadas características em comum.

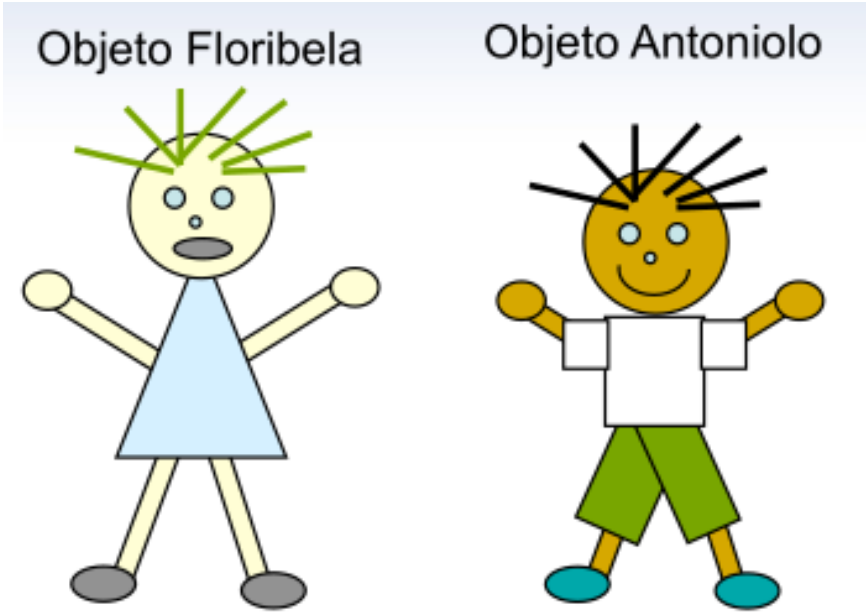
- Ao definir uma classe, então, devemos definir dois pontos principais:

  1 – **atributos**, que são informações da classe (cor do cabelo, altura, etc…)

  2 – **métodos**, que são as ações que podem ser realizadas pelos objetos de cada classe (andar, correr, falar, pensar, etc...)

Orientação a objetos





Floribela e Antoniolo
são **instâncias** da classe Pessoa

Como definir classes em C++?

```
class NomeDaClasse {

    restrição de visibilidade:
        variáveis;
        protótipo de funções;

};
```

```
using namespace std;

class Pessoa {

    public:
        Pessoa();
        ~Pessoa();
        void retorna_idade(int ano);
    private:
        string nome;
        int ano_nascimento;
};
```

Como definir atributos em C++?

- Em C++ a palavra atributo (*attribute*, em inglês) é utilizada para designar um recurso do próprio C++, e não para designar o conceito da palavra atributo de orientação a objetos.

- Atributos do conceito de orientação a objetos são chamados de variáveis em C++. São declarados dentro do escopo da declaração da classe, como no exemplo anterior.

Como definir atributos em C++?

```
using namespace std;

class Pessoa {

public:
   Pessoa();
   ~Pessoa();
   void retorna_idade(int ano);
private:
   string nome;
   int ano_nascimento;
};
```

atributos da classe

Métodos/funções

- São blocos de código encarregados de realizar alguma operação.

- Sintaxe básica no C++:

```
tipo_retorno nome_do_método (parâmetros) {
    //instruções
}
```

  - *tipo de retorno* → tipo de valor devolvido pelo método ou **`void`** (quando não retorna nada).
  - *lista de parâmetros* → dados que alguns métodos precisam receber para usar em suas operações. Exemplos:
    - (**`int`** `quantidade`**`, string`** `nome`)
    - métodos sem parâmetros: ( )

Como criar métodos em C++?

- Exemplos:

```
void nomeClasse::imprime_mensagem(string mensagem) {
    cout << “Mensagem: “ << mensagem;
}
```

```
int nomeClasse::soma(int valor1, int valor2) {
    return valor1 + valor2;
}
```

Método construtor

- Método especial usado para inicializar objetos quando estes são criados.
  - Aloca recursos do objeto
  - Define o estado inicial dos objetos
  - Garante a existência dos atributos

Método construtor

- No C++:
  - recebe o mesmo nome da classe.

```
NomeDaClasse::NomeDaClasse(parâmetros) {
    //inicialização das variáveis/atributos
}
```

  - Exemplos:

```
Pessoa::Pessoa(string novo_nome) {
    nome = novo_nome;
}
```

```
Pessoa::Pessoa() {
    nome = “Fulano”;
}
```

Método destrutor

- Método especial chamado quando a instância do objeto é destruída.
  - Libera recursos do objeto.
  - Permite executar alguma tarefa antes que o objeto seja eliminado.
  - É acionado quando o objeto sai de escopo ou quando é explicitamente destruído por uma chamada *`delete`* ou *`delete[]`* para ponteiros e arrays alocados dinamicamente com o operador *`new`*.

Método destrutor

- No C++:
  - recebe o mesmo nome da classe precedido por um til (~).

```
NomeDaClasse::~NomeDaClasse() {
    //instruções
}
```

  - Exemplo:

```
Pessoa::~Pessoa() {
    cout << “Método destrutor!” << endl;
}
```

Criação de objetos em C++

- É feita da seguinte forma:
  - Cria-se uma variável com o tipo da classe.

```
NomeDaClasse variavel;
```

  - Exemplo:

```
Pessoa p;
```

Chamando/invocando métodos

- Um método é chamado por outro método de outra classe ou da própria classe.

Sintaxe 1: chamada a partir de outra classe

```
identificador_do_objeto.nome_do_método(lista de argumentos);  // método com parâmetros
identificador-do_objeto.nome_do_método();                     // método sem parâmetros
```

Sintaxe 2: chamada de dentro da própria classe

```
nome_do_método(lista de argumentos);  // método com parâmetros
nome_do_método();                     // método sem parâmetros
```

Chamando/invocando métodos

- Exemplo:

```
// outras instruções
. . . . . . . .
f1.setSalario(3500.00);
// instrução seguinte
. . . . . . . .
f2.setSalario(5650.00);
// intrução seguinte
```

```
void Funcionario::setSalario(double novoSalario)
{
    salario = novoSalario;
}
```

Método de configuração de atributos - *setter*

- Troca o valor atual de um atributo por um novo valor, passado por parâmetro.
- Usa o verbo *set* como padrão, seguido do nome do atributo.
- Exemplo:

```
class Funcionario {
    private:
        double salario;
    public:
        void setSalario(double novo_salario);
        double getSalario();
};

void Funcionario::setSalario(double novo_salario) {
    salario = novoSalario;
}
```

Método de acesso a atributos - *getter*

- Um método de acesso é aquele que retorna o valor de um atributo.
- Usa o verbo *get* como padrão, seguido do nome do atributo.
- Exemplo:

```
class Funcionario {
    private:
        double salario;
    public:
        void setSalario(double novo_salario);
        double getSalario();
};


double Funcionario::getSalario() {

    return salario;

}
```

Referências

- Learn C and C++ Programming:
  - https://www.cprogramming.com/